REPORTE DE PRÁCTICA NO. 1

ANÁLISIS DE EXPRESIONES REGULARES

ALUMNO: PALMA VELAZQUEZ ARIADNA BETSABE

Dr. Eduardo Cornejo-Velázquez

1. Introducción

En el presente documento de reporte de practica se puede encontrar el desarrollo que se llevo acabo para poder realizar un codigo que pueda simular el como trabaja la etapa numero uno de un compilador la cual es :*"ANALISADOR LEXICO"*. El objetivo principal de un analizador léxico es transformar un flujo de caracteres de entrada en una secuencia de unidades con significado propio denominadas tokens (como palabras reservadas, identificadores, operadores y símbolos de puntuación).
En este se vera la forma de como realizarlo con codigo de forma clasica y otro en donde se utiliza las expresiones regulares; las cuales provienen de las expresiones formales del lenguaje humano.
Esta ultima forma ayuda a poder optimizar lineas de codigo asi como a declarar de forma mas clara y concisa que es lo que realmente tiene que realizar el codigo desarrollado.
OBJETIVOS DE LA PRACTICA:

- Construir programas en un lenguaje de programación de alto nivel que realicen el análisis de cadenas de símbolos que forman palabras basado en la implementación de estructuras de datos y algoritmos de análisis, además del uso de expresiones regulares como segunda estrategia de implementación.

2. Marco teórico

El proceso de compilación se divide en varias fases, siendo el análisis léxico (o scanning) la primera etapa crítica. En esta fase, el compilador lee el flujo de caracteres del programa fuente y los agrupa en unidades con significado lógico denominadas tokens.

1. **El Analizador Léxico (Lexer)**

El analizador léxico tiene como función principal simplificar la tarea del analizador sintáctico. Al eliminar comentarios y espacios en blanco, y clasificar los elementos del lenguaje (identificadores, palabras reservadas, operadores), reduce la complejidad de la gramática de entrada.

1. **2. Definición y Rol de las Expresiones Regulares (ER)**

Las expresiones regulares son la notación formal estándar para especificar los patrones que describen a los tokens. Su relevancia radica en su capacidad para describir lenguajes regulares, que son los más simples en la jerarquía de Chomsky pero suficientes para la estructura léxica de casi todos los lenguajes de programación.

- *Alfabeto (Σ):* Un conjunto finito de símbolos (caracteres).Lenguaje
- *Regular:* Un lenguaje que puede ser definido por una expresión regular.

Por lo tanto, las palabras del lenguaje estarán formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial de la gramática utilizando las producciones que la definen.

3. Herramientas empleadas

Para poder llevar acabo correctamente la p´ractica uno del curso, se utilizaron las siguientes herramientas:

- Software de edición de codigo (Visual Studio Code)
- Lenguaje de programación (C++)
- Libreria regex
- Regex101
- Computadora

4. Desarrollo

- ***Fase 1: Análisis por Lógica de Control (Código Clásico)***

En esta etapa inicial, el objetivo es comprender la naturaleza de los caracteres a nivel de bits o tablas (como ASCII/Unicode) sin ayuda de herramientas externas.
Estructura del Código

- Modularidad: Cada función tiene una responsabilidad única (ej. esMayuscula, esEntero).
- Lógica Manual: Se utilizan ciclos for para recorrer la cadena y condicionales if-else que evalúan rangos numéricos.
- Ejemplo: Para detectar minúsculas, evalúas si el carácter c está entre 'a' y 'z'.
- Punto de Control (Main): El programa solicita una entrada al usuario y, mediante una estructura de control, imprime mensajes booleanos ("Cumple" / "No cumple") llamando a cada función.
- ***Fase 2: Diseño de Patrones en Regex101***

Antes de programar, es vital validar los patrones. Regex101 funciona como un entorno de depuración para lenguajes regulares.
Pasos en la Plataforma

- Selección del Flavor: Elegir el motor (PCRE, Python o C++ std::regex).
- Pruebas de Carga: Ingresar "Test Strings" que incluyan casos de éxito y casos de fallo (letras dentro de números, espacios, etc.).
- Construcción de Tokens:
- Enteros: "[0-9]"
- Variables: "[a-zA-Z][a-zA-Z0-9]"
- Mayúsculas: "[A-Z]"
- Explicación del Token: Utilizar el panel de "Explanation" para entender qué hace cada cuantificador (como + o *) y los anclajes de inicio (potencia) y fin (signo pesos).

Fase 3: Implementación con Librería regex En la fase final, el código se vuelve mucho más compacto y potente, delegando la lógica de comparación al motor de la librería.

- Características del Nuevo Código
- Librería Especializada: Se incluye ¡regex¿ (en C++).
- Uso de regexmatch: A diferencia de regexsearch, regexmatch asegura que toda la cadena ingresada por el usuario encaje perfectamente con el patrón definido. Si sobra un solo carácter no deseado, retornará false.
- Identificación de Símbolos No Alfanuméricos: Se añade una función específica que utiliza el patrón [a-zA-Z0-9]. Este es un "set negado", lo que significa que buscará cualquier cosa que no sea una letra o un número.

5. Conclusiones

En conclusión esta práctica demuestra la evolución fundamental entre la programación imperativa básica y la capacidad de abstracción avanzada necesaria en el desarrollo de software profesional. Al comenzar con una estructura de control clásica basada en ciclos y condicionales, se logra comprender la mecánica interna del análisis de datos y cómo la computadora evalúa cada bit de información de forma secuencial. Esta etapa es crucial porque establece las bases de la lógica algorítmica y permite valorar la complejidad detrás de tareas que parecen simples, como identificar una variable o un número.
Sin embargo, el salto hacia el uso de Regex101 y la implementación de la librería especializada marca un punto de inflexión donde la eficiencia y la limpieza del código se vuelven prioridades. Se concluye que, mientras el código manual es excelente para entender los fundamentos, las expresiones regulares representan una herramienta superior para el procesamiento de texto, ya que permiten sustituir decenas de líneas de código por patrones compactos y potentes. La inclusión de una función para detectar símbolos no alfanuméricos cierra el ciclo de aprendizaje, demostrando que un buen analizador no solo debe reconocer lo que es válido, sino también identificar y filtrar aquello que no lo es, garantizando así la integridad y seguridad de la información procesada.

Referencias Bibliográficas

References

Hopcroft, J. E., Motwani, R., Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación (3ra ed.). Addison-Wesley. Louden, K. C. (2004). Construcción de compiladores: Principios y práctica. Thomson.